dbt Certification ��� Checkpoint 3: Build Resilient Pipelines at Scale

Checkpoint 3 — Build Resilient Pipelines at Scale (Meta Study Guide)

> Synthesized exam-prep guide for Checkpoint 3. Reorganizes the 19 per-resource cheat sheets into the larger story: **how to design, test, deploy, and recover dbt pipelines at scale without alert fatigue or runaway warehouse cost**.

The shape of Checkpoint 3

CP3 is the operational checkpoint. It bundles four related concerns:

1. **Testing strategy** — what to test, where to put tests, which tools (generic, singular, unit, custom).
2. **State-aware execution** — Slim CI, defer, clone, retry, source-freshness-driven builds.
3. **Discoverability** — exposures and data product surface area.
4. **Project resilience** — the *Essential Checklist*, the structure guide as a foundation.

If CP1 is the mental model and CP2 is the API contract, CP3 is **how to run a dbt project in production without setting your warehouse on fire**.

1. The CP3 worldview

Read first: `01-how-we-structure-overview.md` (cross-reference into CP1).

Everything in CP3 assumes the staging → intermediate → marts structure. State-based selectors, exposures, source freshness, test placement — all rely on the standard folder layout for power. If you skip this, none of the rest works cleanly.

Key reminder of where CP3 artifacts live: | Artifact | Location | |—|—| | Source freshness + tests | `_<dir>__sources.yml` (staging) | | Model data tests | `_<dir>__models.yml` next to model | | Singular tests | `tests/*.sql` | | Custom generic tests | `tests/generic/*.sql` (or `macros/`) | | Unit tests | YAML next to model, **under `models/`** (NOT `tests/`) | | Exposures | `_<area>__exposures.yml` alongside marts | | State (CI artifact) | Prod's `target/manifest.json` + `run_results.json` |

2. Testing strategy — what & where

The “Test smarter” framework

Read in order: `02-blog-test-smarter.md`, `03-blog-test-smarter-where.md`.

Phase 1: identify (three buckets)

- **Data hygiene** — granularity (PK uniqueness, not-null), completeness, formatting. Lives in **staging**.
- **Business-focused anomalies** — relative to known business norms. Threshold drifts; humans set and adjust. Discovered by surveying critical BI dashboards.
- **Stats-focused anomalies** — distributional (volume, dimensional, column outliers). Out of scope until hygiene + business are solid.

Phase 2: prioritize (three filters → severity)

Is the data: - **Customer-facing?** - **Used for financial decisions?** - **Executive-facing?**

If yes to any: **severity = error**. Otherwise: **warn**, or **delete the test entirely**.

> If you don’t have an immediate action when a test fails, it should be a warning. Or removed.

Phase 3: action plan

For every concern, write 1–2 initial debugging steps. Put them in: - A team **testing framework doc** linked from the README. - The test’s own **description** field.

If you can’t articulate an action step → drop the test.

Where each test type lives (the layer map)

Read: `03-blog-test-smarter-where.md` again — the lookup table from this post is the most-asked CP3 question.

Layer	What to test
Sources	Only **fixable-at-source** issues + freshness. If you can’t fix at source, mitigate in staging instead — don’t add a source test.
Staging	Single-table **business anomalies** (accepted ranges, sign invariants, volume bounds). **Don’t test your own cleanup work**.
Intermediate	PK tests on re-grained or enriched models. Anomaly tests on new categorical/range/computed columns. Unit tests for complex transforms.
Marts	Unit tests for complex transformation logic. PK tests where grain changes (or is intentionally preserved). Singular tests for high-impact, high-impact-only-here checks.
CI/CD	Slim CI; all the above tests run automatically.
Advanced CI	Modified/added/removed diff flags as reviewer evidence (dbt Cloud).

Source freshness severity rule

- Source feeds high-impact output → severity = **error** (stale source fails pipeline).
- Otherwise → severity = **warn**.

---

3. The four test families

Read: `08-data-tests.md`, `10-custom-generic-tests.md`, `12-unit-tests.md`, `09-test-source.md`.

Family	Where defined	Runs against	When validated
Generic data tests	YAML `data_tests:` on model/source/seed/snapshot	Real data	After build
Singular data tests	`tests/*.sql` (raw SELECT returning failing rows)	Real data	After build
Custom generic tests	`tests/generic/*.sql` (`{% test name(...) %}`)	Real data	After build
Unit tests	YAML `unit_tests:` under `models/`	Synthetic inputs (given/expect)	**Before** materialization

Pass/fail mechanic (all four)

Test = `select` query returning failing rows. **0 rows = pass.**

Built-in generics

`unique`, `not_null`, `accepted_values`, `relationships`. Plus everything in `dbt_utils` and `dbt-expectations`.

Custom generic essentials

```
-- tests/generic/test_is_even.sql
{% test is_even(model, column_name) %}
    select * from {{ model }} where ({{ column_name }} % 2) = 1
{% endtest %}
```

- Accepts `model` (always called that, even for sources/seeds/snapshots) and optionally `column_name`.
- Add extra args (e.g. `to`, `field` for `relationships`).
- Use `{{ config(severity=...) }}` inside the test for defaults, override per-instance.
- Document the underlying macro with name `test_<name>`.

Unit test essentials

```
unit_tests:
  - name: test_is_valid_email_address
    model: dim_customers
    given:
      - input: ref('stg_customers')
        format: dict
        rows: [{email: cool@example.com, ...}]
    expect:
      format: dict
      rows: [{email: cool@example.com, is_valid_email_address: true}]
```

- v1.8+ feature. Live in YAML next to the model under `models/`.
- Three fixture formats: `dict` (inline), `csv` (inline or file), `sql` (inline or file).
- Direct parents must exist in the warehouse before running the test (use `--empty` to bootstrap).
- `dbt build` flow: unit tests → materialize → data tests. Warehouse spend is skipped if unit tests catch the bug.
- **Don’t run in production** — `--exclude-resource-type unit_test`.
- For incrementals: expected output = the merge/insert result, not the final table.

Source testing nuance

Same toolkit as model tests, attached to source YAML. Only test **fixable-at-source** problems. Mitigate everything else in staging.

---

4. Data test configurations

Read: `11-data-test-configs.md`.

Three places to configure, most-specific wins

1. **Property YAML** (next to the test instance)
2. **`config()` in test SQL** (the test definition or singular test)
3. **`dbt_project.yml`** under `data_tests:`

Singular tests can’t be configured from property YAML — only their own `config()` or the project file.

Configs you’ll use

- `severity: error | warn` + `error_if`/`warn_if` thresholds (`">0"`, `">10"`).
- `where:` — filter the rows the test runs over (huge cost win on big tables).
- `store_failures: true` (+ `store_failures_as`, `database`, `schema`, `alias`) — persist failing rows.
- `fail_calc: "<sql>"` — override how failures are counted.
- `limit: <int>` — cap failing rows returned.
- General: `enabled`, `tags`, `meta`.

Killer recipe: cheap test on huge table

```
- unique:
    config:
      where: "order_date >= dateadd('day', -30, current_date)"
```

Store failures pattern

```
dbt test --store-failures
```

Failures land in `dbt_test__audit` schema; replaced on each run.

---

5. State-aware execution

This is the half of CP3 most people skip and then suffer for.

Read in order: `17-state-selection.md`, `18-state-method.md`, `13-cmd-test.md`, `19-cmd-retry.md`, `06-cmd-clone.md`, `07-clone-incremental.md`, `05-blog-defer-or-clone.md`.

State artifacts (recap from CP2’s compile)

- `manifest.json` — current/state project structure.
- `run_results.json` — node-level results of last invocation.
- `sources.json` — output of `dbt source freshness`.

`--state` is the gateway

Pass `--state path/to/manifest_dir` to enable: - `state:` selectors (modified, new, old, modified.body, etc.) - `result:` selectors (pass/fail/error/warn/skipped from last run) - `source_status:` (fresher+, etc.) - Deferral (`--defer`) - `dbt clone`

Slim CI — the canonical pattern

```
dbt build --select state:modified+ --defer --state path/to/prod
```

- Builds only modified models + their descendants.
- For unselected models, `ref()` resolves to production via `--defer`.
- Cheap, fast, mimics post-merge behavior.

The state selectors you must know

Selector	Selects
`state:new`	New since state
`state:modified`	Changed since state
`state:modified.body`	SQL body changes only
`state:modified.configs`	Config changes only
`state:modified.relation`	Materialization/alias changes
`state:modified.contract`	Contract changes
`state:old`	Already exists in state
`state:unmodified`	Opposite of modified
`result:fail` / `result:error` / `result:pass` / `result:warn` / `result:skipped`	From `run_results.json`
`source_status:fresher+`	Sources fresher than the prior `sources.json`

Defer vs clone — quick decision

	defer (`--defer`)	clone (`dbt clone`)
Creates objects?	❌	✅ (real warehouse objects)
BI-visible?	❌	✅
Safely mutate target?	❌	✅
Multi-source?	✅	❌ (one src schema → one target)
Drift?	Always latest	Point-in-time
Typical use case	**Slim CI**	**CD / blue-green / sandboxes**

Rule of thumb: **defer for CI, clone for CD**.

The clone-incremental recipe (most-loved CP3 trick)

```
        # Step 1: clone modified incrementals (+ downstream) that already exist in
prod
        dbt clone --select state:modified+,config.materialized:incremental,state:old \

                  --state path/to/prod

        # Step 2: Slim CI as normal — incrementals now exist in PR schema, so
is_incremental() is true
        dbt build --select state:modified+ --defer --state path/to/prod
```

Why `state:old`? Filters out brand-new incrementals that don't exist in prod yet to clone — they should run full-refresh anyway.

`dbt retry`

Resumes the prior invocation from the point of failure using `run_results.json`. Idempotent — same bug = same failure. Fix the bug, then retry runs only what's still failing/skipped.

Works for: `build`, `compile`, `clone`, `docs generate`, `seed`, `snapshot`, `test`, `run`, `run-operation`.

Core inherits selection from prior run. Fusion lets you override with `--select` / `--exclude`.

Pairs especially well with **microbatch** incremental models — retries failed batches independently rather than the whole model.

`dbt source freshness` and `source_status:fresher+`

Read: `16-cmd-source.md`.

```
        dbt source freshness                                    # snapshot

        dbt build --select source_status:fresher+ --state path/to/prod  # rebuild
downstream of fresh sources
```

Set `error_after` for high-impact sources; `warn_after` only for nice-to-knows.

`dbt test` CP3 angle

Read: `13-cmd-test.md`.

Most-used selectors in CP3: - `test_type:data` / `test_type:unit` / `test_type:generic` / `test_type:singular` - `result:fail` / `result:error` - `state:modified` - `source_status:fresher+` - `+exposure:<name>`

⚠️ `dbt test` overwrites `run_results.json` — for combined `result:fail` + `result:error` selectors, use one `dbt build` invocation.

---

6. Exposures — make consumers visible

Read: `14-exposures.md`, `15-exposure-properties.md`.

An exposure declares a downstream use (dashboard, ML model, app, notebook, analysis) as a first-class node in the DAG.

Minimum viable exposure

```
        exposures:
          - name: weekly_jaffle_metrics
            type: dashboard
            owner: { email: data@jaffleshop.com }
            depends_on: [ ref('fct_orders'), ref('dim_customers') ]
```

Five `type` values

`dashboard`, `notebook`, `analysis`, `ml`, `application`.

Why exposures matter for resilience

- `dbt build --select +exposure:executive_summary` — rebuild & test everything an exposure depends on.
- The exposure appears in docs/Catalog (orange **EXP** badge in the DAG).
- Health tiles let BI users see freshness/test status at a glance.
- "Test smarter — where" uses exposures to identify high-impact pipelines for stricter testing.

v1.10+ structure

`tags` and `meta` moved under `config:`:

```
        - name: my_exposure
          config:
            tags: ['critical']
            meta: {tier: 1}
            enabled: true
```

Project-level toggle

```
        exposures:
          +enabled: true
```

---

7. The Essential Project Checklist

Read: `04-blog-essential-checklist.md`.

A field-tested audit pass for any 2+ month-old dbt project. Run through it quarterly. Categories:

- **dbt_project.yml** — meaningful name, no redundant `materialized: view`, folder-level materializations, `on-run-end` grants, sparing tags, YAML selectors.
- **Package management** — `packages.yml` up-to-date; `dbt_utils` installed.
- **Code style** — documented + enforced.
- **Project structure** — staging/marts present, prefixes (`stg_`, `fct_`, `dim_`), modular, filter early.
- **dbt usage** — recent version; know longest-running models; sources used everywhere; tests in dev *and* prod; Jinja readable; incrementals use `unique_key` + `is_incremental()`.
- **Testing & CI** — 100% test coverage ideal; minimum PK uniqueness + not-null; PRs with mandatory review.
- **Documentation** — descriptions on every model; doc blocks for column descriptions; current README.
- **dbt Cloud** — jobs inherit env dbt version; CI job exists; run cadence matches source cadence; Slack notifications.

---

8. Putting CP3 together — end-to-end resilience

Imagine a production dbt project with daily `dbt build` jobs, Slim CI, an executive dashboard exposure, several incremental models, and source-freshness SLAs.

Daily prod workflow

1. **Source freshness snapshot** (every 30 min): `dbt source freshness`.
2. **Hourly rebuild**: `dbt build --select source_status:fresher+ --state path/to/prod`.
3. **Build alerts**: failed test severity = `error` → pipeline fails → ops gets paged.

When a build fails

1. **Diagnose**: `dbt show --select <failing_test>`; read `target/run/<model>.sql`; check `dbt.log`.
2. **Fix code**, push commit.
3. **Resume**: `dbt retry` picks up from the failure point.

PR workflow

1. Slim CI clones modified incrementals first:

```
        dbt clone --select
state:modified+,config.materialized:incremental,state:old --state path/to/
prod
```

2. Then runs:

```
        dbt build --select state:modified+ --defer --state path/to/prod
```

3. Unit tests catch logic bugs before materialization.
4. Failed tests block the PR.

Test placement during the PR

- Source tests catch fixable issues at the boundary.
- Staging tests catch business anomalies on single tables.
- Intermediate tests cover new joins/calculations and grain changes.
- Marts tests cover the calculated fields and any singular high-impact checks.
- Unit tests cover complex transformation logic — they're cheap (synthetic data).

Sandbox for analysts

1. Need a fresh prod-shaped dataset to experiment with: `dbt clone --select <whatever> --state path/to/prod`.
2. Now analysts can mutate the sandbox without touching prod (and the BI tool can read it).

Quarterly hygiene

Run the Essential Checklist; close gaps as tickets.

Exam-ready summary tables

Test family quick map

Family	Where	Inputs	When validated
Generic data	YAML `data_tests:`	Real data	Post-build
Singular data	`tests/*.sql`	Real data	Post-build
Custom generic	`tests/generic/*.sql` (or `macros/`)	Real data	Post-build
Unit	`unit_tests:` YAML under `models/`	Synthetic	Pre-materialization

Severity decision (per "Test smarter")

Concern feeds…	Severity
Customer / financial / executive	`error`
Future research, within 6 months	`warn`
Anything else	no test

State selector → use

Selector	Use
`state:modified+`	Slim CI scope
`state:old`	Clone-incremental filter
`result:fail`	Rerun failed tests
`result:error+`	Rerun errored nodes + downstream
`source_status:fresher+`	Rebuild downstream of fresh sources
`+exposure:<name>`	Build everything a downstream consumer needs

Defer vs clone

Need	Use
Slim CI / cheap CI	**defer**
Object in BI tool / mutable sandbox / blue-green	**clone**
Test incrementals as incremental in CI	**clone** (then defer)

Tests where to put them (layer map)

Layer	Tests
Source	Fixable-at-source; freshness
Staging	Business anomalies (single-table)
Intermediate	New columns; re-grained PKs; unit tests for complex SQL
Marts	New calculated fields; unit tests; singular high-impact tests
CI/CD	All the above on `state:modified+ --defer`

Resilience commands

Command	Purpose
`dbt build`	Run + test in DAG order; production default
`dbt source freshness`	Check source SLAs
`dbt clone`	Copy schemas; CD use cases
`dbt retry`	Resume failed invocation from point of failure
`dbt show`	Preview failing rows / inspect model

Exposure types

Type	What it represents
`dashboard`	A BI dashboard
`notebook`	A Jupyter / similar analysis
`analysis`	One-off analytical write-up
`ml`	ML model or pipeline
`application`	App consuming dbt-produced data

How to study from here

1. Walk through the "PR workflow" and "production failure" stories in §8 mentally.
2. Memorize the testing-layer map and the severity decision tree.
3. Know defer vs clone cold — they show up in scenarios.
4. Know the `state:` and `result:` selector names and what each picks.
5. Be able to write the clone-incremental + Slim CI two-command CI job from memory.
6. Run through the Essential Checklist on a project you know — note 3 things you'd change.

File index (this directory)

- `01-how-we-structure-overview.md`
- `02-blog-test-smarter.md`
- `03-blog-test-smarter-where.md`
- `04-blog-essential-checklist.md`
- `05-blog-defer-or-clone.md`
- `06-cmd-clone.md`
- `07-clone-incremental.md`
- `08-data-tests.md`
- `09-test-source.md`
- `10-custom-generic-tests.md`
- `11-data-test-configs.md`
- `12-unit-tests.md`
- `13-cmd-test.md`
- `14-exposures.md`
- `15-exposure-properties.md`
- `16-cmd-source.md`
- `17-state-selection.md`
- `18-state-method.md`
- `19-cmd-retry.md`

How We Structure Our dbt Projects (CP3 Reading)

Source: https://docs.getdbt.com/best-practices/how-we-structure/1-guide-overview
Type: Reading · Checkpoint 3 (cross-referenced from Checkpoint 1)

> ℹ️ This resource also appears in Checkpoint 1. The full cheat sheet lives at `checkpoint-1/02-how-we-structure-overview.md` (plus the four layer-specific sheets `03–06`). This file highlights what Checkpoint 3 cares about most: where tests, exposures, and other resilience artifacts fit in the structure.

Why CP3 sends you back to this guide

Checkpoint 3 is about **resilient pipelines** — testing strategy, exposures, defer/clone, source freshness, state, retries. None of that lands cleanly unless you can name *where each artifact lives* in the canonical staging → intermediate → marts shape.

The structure (recap, abridged)

```
models/
├── staging/<source_system>/
│   ├── stg_<source>__<entity>s.sql
│   ├── _sources.yml          ← source-level tests, freshness, exposures, event_time
│   ├── _models.yml           ← staging-model tests
│   └── base/                 ← only when joins/unions are needed pre-stage
├── intermediate/<area>/
│   └── int_<entity>_<verb>s.sql
└── marts/<department>/
    └── <entity>.sql          ← table or incremental; consumers expect contracts
```

Plus: - `tests/` — singular tests (cross-model logic that generic tests can't express) - `snapshots/`, `seeds/`, `analyses/`, `macros/` - A `_<dir>__models.yml` per folder with model configs/tests

Where CP3 artifacts live

Artifact	Lives where
Source freshness	`_sources.yml` under `freshness:` and `loaded_at_field`
Source data tests	`_sources.yml` columns (`unique`, `not_null`, ranges)
Generic data tests	`_<dir>__models.yml` under each model's `columns:`
Singular tests	`tests/` directory; one `.sql` per test
Custom generic tests	`tests/generic/` directory; macros named `test_<name>`
Unit tests	YAML next to the model (`models/.../*.yml`)
Exposures	YAML in the relevant area (often `models/marts/<area>/_<area>__exposures.yml`)
Snapshots	`snapshots/` (or `models/` per v1.9 YAML config)
State artifacts	Outside the project — production `target/` from a previous run, referenced via `--state path/to/prod`

Selection patterns that depend on this structure

The whole point of the folder hierarchy is **node selection power**. CP3 commands rely on it constantly:

```
# Run all tests on staging Stripe data
dbt test --select staging.stripe

# Test only what changed (Slim CI)
dbt build --select state:modified+ --defer --state path/to/prod

# Source-freshness-driven build
dbt source freshness
dbt build --select source_status:fresher+ --state path/to/prod

# Rerun only failed batches/tests
dbt retry

# Build everything downstream of an exposure
dbt build --select +exposure:my_dashboard
```

Each pattern selects by folder, state, source status, or graph operator — these only work if the structure is consistent.

Why CP3 keeps quoting this guide

- "Test smarter not harder" tells you *what* tests to write. **Structure** tells you *where* to put them.
- "Where should tests go in your pipeline?" maps test types to layers — that mapping only makes sense with this structure in mind.
- Defer/clone, state selection, source freshness — all rely on a project organized so that `--select state:modified+` is meaningful.

Key takeaways

- The structure isn't decorative — CP3's selection, defer, and CI workflows depend on it.
- Tests live close to the resource they validate: source tests in `_sources.yml`, model tests in `_<dir>__models.yml`, unit tests next to the model, singular tests in `tests/`.
- Exposures sit alongside their owning marts so node selection can find them via `+exposure:name`.
- State-comparison patterns (`--state`, `--defer`) treat the structured folder layout as their stable contract.

Related

- Layer-specific guides (staging / intermediate / marts / rest-of-project) in CP1.
- Test-smarter (CP3) — testing by layer.
- Slim CI / state selection / defer / source-freshness selectors.

Test Smarter Not Harder

Source: https://docs.getdbt.com/blog/test-smarter-not-harder **Type:** Reading · Checkpoint 3

What it is

A 2024 dbt Labs post that proposes a **testing framework** for dbt projects. The thesis: testing should drive *action*, not accumulate alerts. Bad testing has two failure modes — too few tests (only PKs), or too many (alert fatigue). This guide threads the middle path.

The two-phase framework

Phase 1: Identify

Bucket data-quality issues into three categories.

Bucket 1: Data hygiene

Issues you fix in the **staging layer**. The data should meet formatting, completeness, and granularity expectations. - **Granularity** — primary keys unique and not null. - **Completeness** — columns that should contain text always do. - **Formatting** — email addresses have valid domains, etc.

Bucket 2: Business-focused anomalies

Unexpected behavior relative to known business norms. Human-defined; thresholds drift over time and need updating.

Discovery method: pick 1–3 frequently used BI dashboards/tables. For each, list 1–3 expected behaviors. Examples: - Revenue shouldn't move >X% in Y time (fraud / OMS signal). - MAU shouldn't decline >X% post-onboarding (UX signal). - Exam pass rate shouldn't dip below Y% (content/tech signal). Also: mine recent data incidents (#data-questions, stakeholder DMs) for 3–4 prior issues to guard against.

Bucket 3: Stats-focused anomalies

Distributional fluctuations: volume anomalies, dimensional anomalies (too many products under a line), column anomalies (values >N stdev from mean). Generally requires more advanced tooling — out of scope of this post. Tackle after hygiene + business anomalies are solid.

Phase 2: Prioritize

For every concern, ask whether the affected data is: - **Customer-facing?** - **Used for financial decisions?** - **Executive-facing?**

These are **high-impact, pipeline-failing** events → severity = **error**.

Everything else is **nice-to-know** → severity = **warn**, or remove entirely. *"If you don't have a specific action you can immediately take when a test fails, the test should be a warning, not an error."* Kept-but-warned tests are only justified if you're using them to inform a decision in the next ~6 months.

Action plan

For each prioritized concern, write down **1–2 initial debugging steps**. Examples: > Revenue shouldn't change by more than X% in Y time. > - Check recent revenue values in staging model. > - Identify transactions near min/max values. > - Discuss outliers with sales-ops team.

Add these to: - A team **testing framework doc** (linked in project README). - The **test's description** (yes, singular tests can have descriptions).

If you can't define an action step, *delete the test* (or move it to a backlog).

Recommended packages

After your prioritized list is done, find tests on `hub.getdbt.com`: - `dbt-expectations` - `dbt_utils`

What makes a "good" data quality program

Trusted, frequently used data. No more "let me re-derive the metric myself to double-check." Testing exists to deliver that trust — not to fire alerts into Slack at 3am that everybody mutes.

Severity, recap

- **error** — pipeline fails. Customer-facing / financial / executive-impacting concerns only.
- **warn** — pipeline continues; result shows up in `run_results.json` and dashboards. Use for genuinely useful informational checks tied to upcoming work.
- **no test** — when you can't articulate the action.

Tools worth knowing about

- **dbt Advanced Testing course** — deeper coverage.
- **`dbt_meta_testing`** / **`dbt_project_evaluator`** — enforce best practices.
- **dbt Explorer Recommendations** — surfaces missing tests, anti-patterns.

Key takeaways

- Three buckets: hygiene, business anomalies, stats anomalies.
- Three prioritization questions: customer-facing? financial? executive?
- Tests without actionable debugging steps → drop or downgrade to warn.
- Living framework document, linked in README.
- Pair this *what to test* with the next post (*where to put tests*).

Related

- *Test smarter — where should tests go?* (the layered placement guide).
- Data tests, custom generic tests, unit tests.
- `severity`, `store_failures` configs.
- Advanced Testing course; `dbt-expectations`, `dbt_utils`.

Test Smarter — Where Should Tests Go?

Source: https://docs.getdbt.com/blog/test-smarter-where-tests-should-go **Type:** Reading · Checkpoint 3

What it is

The follow-up to *Test smarter not harder*. Once you've prioritized **what** to test, this post tells you **where** in the pipeline each test belongs — mapped to dbt Labs' canonical staging / intermediate / marts / CI structure.

The headline rules

- **Sources** — tests should catch **fixable-at-the-source-system** issues.
- **Staging** — business-focused anomalies on individual tables; clean up nulls/dupes/outliers that *can't* be fixed at source.
- **Intermediate & marts** — anomaly tests resulting from joins/calculations + extra primary-key/not-null where grain protection matters.
- Don't re-test passthrough columns at every layer.
- Don't add tests that validate your own cleanup work (e.g., `not_null` on a column you just filtered nulls from).

"Fixable" defined

A source issue is fixable if either: - *You* can correct it at the source system, OR - You know the right person and have a strong-enough relationship to actually get it fixed.

If a problem is technically fixable but won't realistically happen this planning cycle → mitigate in **staging**, not via a source test that just rings the alarm bell.

Per-layer guidance

Sources

Source freshness - Sources feeding **high-impact** outputs (customer-facing / financial / executive) → `dbt source freshness` in your job, severity = **error**. Stale source = failed job. - Sources NOT feeding high-impact outputs → severity = **warn**. You still get visibility; pipeline keeps running.

Source data tests (must be source-fixable) - Duplicate records that can be deleted at the source. - Nulls that can be filled in at the source (e.g., missing customer email). - PK uniqueness when duplicates are removable upstream.

Staging

- **Data cleanup** — use the standard staging best practices (rename, recast, categorize). **No test on your own cleanup**: don't write `not_null` on a column whose nulls you're filtering out.
- **Business anomaly tests** — single-table, define expected boundaries:
  - Accepted ranges (e.g., a store can't sell more units than it received).
  - Sign expectations (negative transactions only on returns).
  - Volume bounds outside seasonal norms.

Intermediate (when present)

Focus on **new** columns and **grain changes**. - **Primary key tests** on any model that re-grains. - PK tests on enriched-but-same-grain models too — documents intent for future devs. - After joins/aggregations, simple anomaly tests: - `accepted_values` on new categorical columns. - `mutually_exclusive_ranges` (`dbt_utils`) for related range columns (e.g., age brackets). - `not_constant` (`dbt_utils`) for columns that should always change (e.g., page-view counters). - Consider **unit tests** for particularly complex SQL.

Marts

Same hygiene-or-anomaly pattern, focused on new columns. - **Unit tests** for transformation logic with branching/CASE-WHEN (forecasting dates, customer segmentation). - **PK tests** where mart grain differs from upstream — or where grain matches but intent should be documented. - **Business anomaly tests** on new calculated fields: - Singular tests for specific high-impact concerns (e.g., fuzzy-matching to detect free-trial abuse). - "This calculation can't move by more than X% week-over-week." - Ledger-style invariants ("today's running total ≥ yesterday's").

CI/CD

This is where the testing framework gets **automated**. - Use **Slim CI**: `dbt build --select state:modified+ --defer --state path/to/prod`. - All your prioritized, well-placed tests run on every change, but only against modified models and their descendants. - With CI + scheduled prod runs, the framework is on autopilot.

Advanced CI (dbt Cloud)

- Start with the built-in **modified / added / removed** flags in the "compare changes" UI.
- These are gut-check evidence for reviewers — easy review baseline.
- More advanced row-level diffing is available but goes beyond the post's scope.

Mental model

> Testing is like marathon training. Random testing = running 20 miles a day with no plan and getting injured. Smart testing = a training plan, revised as your needs evolve.

Key takeaways

- Source tests guard things you can fix at the source — everything else moves to staging.
- Staging fixes data and tests **business anomalies on single tables**.
- Intermediate/marts test **new** logic only — don't retest passthroughs.
- PK tests where grain changes OR where enrichment happens (intent signaling).
- Unit tests at the layer doing complex transformations (often marts, sometimes intermediate).
- Slim CI automates the framework; Advanced CI gives reviewer evidence.

Related

- *Test smarter not harder* (the prerequisite framework post).
- *How we structure our dbt projects* (the layer definitions this post depends on).
- Data tests, unit tests, custom generic tests, source freshness.
- Slim CI / state selection.

Your Essential dbt Project Checklist

Source: https://docs.getdbt.com/blog/essential-dbt-project-checklist **Type:** Reading · Checkpoint 3

What it is

A field-tested audit checklist for cleaning up dbt projects. After seven client audits, dbt Labs found the same opportunities in every project: better performance, better maintainability, easier onboarding. Use this list to score your own project.

The audit checklist (compressed)

dbt_project.yml

- **Project name** is meaningful (`my_company_analytics`, not `my_new_project`).
- No `materialized: view` clutter — that's the default; don't declare it.
- Set materializations at the folder level when whole subdirs share one, not per-model.
- Strip placeholder comments from `dbt init`.
- Use `on-run-end` post-hooks to **grant** permissions to BI/analyst roles.
- Use **tags sparingly** — they should mark exceptions, not enumerate the norm. Folder selectors usually replace tag-based selection.
- Consider **YAML selectors** for complex multi-criteria selection — they read better than chained tag expressions.

Package management

- `packages.yml` versions are current — check against `hub.getdbt.com`.
- `dbt_utils` installed. It's the de facto standard kit.

Code style

- Documented, **enforced** style (SQL formatting, field naming).
- Make use of window functions and aggregations instead of manual loops/CTEs.

Project structure

- Staging / intermediate / marts present, with `stg_`, `int_`, `fct_`/`dim_` prefixes.
- One transformation per model; one transformation per CTE.
- **Filter early** — pushing filters upstream is the most common missed optimization.
- Macro file names clearly indicate the macros inside.

dbt

- Recent dbt version. The further behind, the more painful upgrading becomes.
- Know your longest-running models — are they good candidates for incremental?
- Every model runs in every run? Circular references? Models defined but never built are a smell.
- Sources are used everywhere; **no model selects from raw tables**.
- Source freshness configured for critical sources.
- `dbt test` runs in dev *and* in production jobs.
- Jinja is readable: `set` statements at the top, whitespace tidy (`{{ this }}` not `{{this}}`), in-line comments via `{# #}`.
- Incremental models use `unique_key` + `is_incremental()` guard.
- Tags exist for a reason and get used; otherwise delete them.

Testing & CI

- ✅ **100% test coverage** as an ideal. Minimum: every model has `not_null` + `unique` on the primary key.
- Tests reflect real assumptions — both about source data and about business logic.
- All changes flow through **PRs with mandatory review**.
- PR template in place.

Documentation

- Every model has a description; complex transformations have explanations.
- Column-level descriptions use **doc blocks** (`_<dir>__docs.md`).
- README exists and is current.
- Onboarding a new person should not require tribal knowledge.

dbt Cloud specifics

- Jobs inherit dbt version from the environment (eases upgrades).
- Jobs make sense — no orphan/unused projects.
- Run frequency matches data update frequency (don't run hourly when source updates daily).
- Periodic full-refresh of production exists.
- Scheduled tests run regularly.
- Longest-running jobs identified; CI job exists (GitHub).
- dbt Cloud has its own warehouse user with a defined default warehouse/role.
- Slack/email notifications on failed jobs are wired up.

Why this matters in CP3

CP3 is about pipelines that survive: tests where they belong, CI that catches what should be caught, deploys that are repeatable. This checklist is the *operational* counterpart to "test smarter" — it surfaces structural debts that make resilience harder.

How to use the checklist

- Print it / save it as a markdown file in the repo.
- Run through it section-by-section quarterly.
- Treat each ❌ as a discrete refactor; file a ticket per one.
- The points marked "default" or "should not" (`materialized: view` declarations, tagging every model) are the cheapest wins.

"Where's Waldo"-style framing

The original post calls this a Waldo book — it tells you what to look for, but you still have to find Waldo in your project. The value is in the **named patterns** of decay, not the search itself.

Key takeaways

- Naming, tagging, and macro hygiene drift fastest — audit them first.
- Sources + freshness + `is_incremental()` are non-negotiable for resilient pipelines.
- Tests: minimum PK uniqueness/not-null on every model; ideal is 100%.
- CI + PR review + doc blocks make on-boarding linear instead of tribal.
- dbt Cloud audits: env-inherited dbt version, scheduled tests, run cadence matching source cadence.

Related

- *Test smarter not harder* / *Where should tests go?*
- *How we structure our dbt projects*.
- `dbt_project_evaluator` and `dbt_meta_testing` packages — automate the checklist.
- Slim CI, `--defer`, `state:modified+` for CI economy.

To Defer or to Clone, That Is the Question

Source: https://docs.getdbt.com/blog/to-defer-or-to-clone **Type:** Reading · Checkpoint 3

What it is

A dbt Labs blog post comparing **defer** and **clone** — two ways to avoid expensive recomputation in dev/CI/CD. Same purpose at a high level, very different mechanics and effects.

Definitions (at a glance)

- `dbt clone` — Uses the warehouse's native zero-copy clone to copy an entire schema (or selected resources) from a source schema to a target schema, almost instantly and almost free. Creates real database objects in the target schema.
- `--defer` **(with** `--state`**)** — At compile time, dbt overrides `ref()` resolutions for unbuilt models to point at the production schema's existing objects. No new objects are created.

How clone works (mechanically)

Zero-copy cloning copies *metadata only* — the underlying data stays at rest. You end up with two pointers (source and target schemas) to the same underlying storage. After clone, you can read, modify, even write to the target schema; the source is untouched.

How defer works (mechanically)

dbt compares two manifests (a "state" path and the current project). For every model **not selected** in the current run, dbt resolves `ref()` to point at the production object instead of trying to build it. The CI job only builds the models that changed.

Defer vs clone — first-order

	defer	clone
How used	`--defer --state path/` flag	`dbt clone` command
Creates objects?	No (just rewires refs)	Yes (real objects in target schema)
Mechanism	Manifest diff; ref overrides	Zero-copy (or pointer view fallback)

Defer vs clone — second-order

	defer	clone
Usable in BI / downstream tools?	❌ Only inside dbt	✅ Anywhere
Safely modify target schema?	❌ Would mutate prod	✅ Yes — sandbox
Drift from source?	❌ Always latest	✅ Point-in-time snapshot
Multiple sources?	✅ Yes (dynamic per-model)	❌ One source schema → one target

When to use which

- **CI** (Slim CI) → **defer**.
  - Avoid rebuilding unchanged models.
  - Reference prod for upstream, staging for the modified subset.
- **Sharing dev data with BI/analysts** → **clone**.
  - They need actual database objects to query.
- **Sandbox experimentation on prod-shaped data** → **clone**.
  - You can mutate it without touching prod.
- **Blue/green deployments** → **clone**.
  - Build full staging dataset, run tests, clone to prod only on green.
- **Testing modified incremental models in CI** → **clone** of the incremental table into the CI env, then re-run only the modifications. See *Clone incremental models* best-practice doc.

Rule of thumb

> **Defer** fits CI use cases. **Clone** fits CD (continuous deployment) use cases.

Worked example: Slim CI

1. Take a known-good `manifest.json` from production (saved as an artifact).
2. In CI: `dbt build --select state:modified+ --defer --state path/to/prod-manifest`.
3. CI builds only what changed; everything else `ref`'s the prod objects.

Worked example: blue/green with clone

1. Build all models into a `staging` schema.
2. `dbt test` against staging.
3. If green: `dbt clone --select <everything>` into prod.
4. If red: do nothing; prod is untouched.

Limitations / gotchas

- **Clone** requires warehouse support for zero-copy clone (Snowflake, BigQuery, Databricks, etc.). On unsupported platforms, dbt falls back to **pointer views** (`select * from src`) — slower for downstream queries.
- **Clone drifts** the moment the source moves on. Re-clone on a schedule if you need fresh sandboxes.
- **Defer** can confuse new contributors — the dev environment "magically" shows prod data via the override. Document it.
- **Defer doesn't materialize anything**. If your downstream tool (Tableau, Looker) needs a real table, you need to clone, not defer.

Memorable framing from the post

> If you link to this page, you **defer** to it. If you print it and write notes in the margins, you've **cloned** it.

Key takeaways

- Both save warehouse cost by reusing previously-built objects.
- **Defer** = lazy ref-override for CI; no new objects, always-latest, multi-source.
- **Clone** = eager metadata copy for sandboxes/CD; real objects, can mutate, point-in-time.
- CI workflow → defer. CD/sandbox/blue-green → clone.
- Pair them in the same project for different stages of the deployment lifecycle.

Related

- `dbt clone` command.
- `--defer` / `--state` / `state:modified+`.
- *Clone incremental models* best practice.
- Slim CI workflows.

`dbt clone`

Source: https://docs.getdbt.com/reference/commands/clone **Type:** Documentation · Checkpoint 3

What it does

Clones selected nodes from a **specified state** to the current target's schema(s). Uses the warehouse's zero-copy cloning where available; falls back to pointer views otherwise.

How it picks a mechanism

- **Zero-copy clone** (Snowflake, BigQuery, Databricks) when the source object exists as a table.
- **Pointer view** (`select * from src_schema.tbl`) when zero-copy isn't supported or the object is a view.

The materialization name is `clone` — dbt internally treats it like any other materialization but it's only used by this command.

Use cases

- **Blue/green continuous deployment** — build to staging, test, then clone to prod.
- **Cloning production into dev schemas** — give analysts/devs a fresh sandbox.
- **Incremental models in CI** — clone the prod incremental table into the CI schema so `is_incremental()` is `true` and CI doesn't do an expensive full-refresh.
- **Testing code changes against downstream BI** — clone produces real objects a BI tool can read; defer cannot.

Common commands

```
# Clone everything from a saved state into target schema(s)
dbt clone --state path/to/artifacts

# Clone just one model
dbt clone --select "one_specific_model" --state path/to/artifacts

# Re-clone (overwrite pre-existing target relations)
dbt clone --state path/to/artifacts --full-refresh

# Speed up with more threads — clone statements are independent
dbt clone --state path/to/artifacts --threads 50
```

By default `dbt clone` will **not overwrite** pre-existing relations in the target — pass `--full-refresh` to force re-clone.

Clone vs defer (recap)

- `dbt clone` creates real objects → usable in BI, sandbox, mutable. Costs some compute and storage metadata.
- `--defer` rewires refs at compile time → no new objects, always-latest, multi-source, but only visible inside dbt.

When in doubt for **CI** → defer. For **CD** or BI-visible sandboxes → clone.

Clone in dbt Cloud

- **Cloud CLI**: `dbt clone` auto-includes `--defer` for you.
- **Studio IDE**:
  1. Have a successful production-environment job run (needed for state).
  2. Toggle **"Defer to production"** in the bottom-right of the command bar.
  3. Run `dbt clone`.

Selection patterns

```
        # Clone all incremental models that are modified OR downstream of a
modification,
        # but only if they already existed in prod (state:old)
        dbt clone --select state:modified+,config.materialized:incremental,state:old \

                  --state path/to/prod
```

This is the canonical pattern from *Clone incremental models* — see that doc for the full CI recipe.

What's actually happening

Zero-copy clone copies *metadata*: the warehouse creates a pointer from the new relation to the same underlying data blocks. Storage cost is near-zero until the clone is mutated (then the warehouse copy-on-writes the changed blocks).

Pointer views (the fallback) are real views — they query through to the source object. Mutation of the source is reflected in the clone; mutation of the clone is not possible (it's a view).

Gotchas

- Clone is **point-in-time**. Data drifts after the operation. Re-clone to refresh.
- Clone of an **incremental** model: the underlying table is cloned, but `is_incremental()` only returns `true` if the *current target* sees the cloned object as existing — which it now does, so subsequent `dbt run` on that model runs incrementally.
- `--full-refresh` re-clones (overwrites), not "refresh the cloned data from source."
- Not all object types clone the same way; views always end up as pointer views.

Key takeaways

- `dbt clone` = zero-copy clone (or pointer-view fallback) of selected nodes from a state to the target schema.
- Use when you need **real objects** in the warehouse — defer doesn't give you that.
- Default behavior preserves existing target relations; `--full-refresh` overwrites.
- Cloud CLI auto-defers; Studio IDE requires the defer toggle.
- The killer recipe: clone prod incrementals into CI before `dbt build --select state:modified+`.

Related

- *Clone incremental models* (the CI recipe).
- *To defer or to clone* (the conceptual blog post).
- `--state`, `state:modified`, `state:old` selection.
- `--defer` / `--state` for the CI alternative.

Clone Incremental Models in CI

Source: https://docs.getdbt.com/best-practices/clone-incremental-models **Type:** Documentation · Checkpoint 3

The problem this solves

You have a Slim CI job:

```
        dbt build --select state:modified+ --defer --state path/to/prod
```

This builds modified models (and descendants) into a **PR-specific schema**. But what about incremental models?

When the CI run hits a modified **incremental** model, the model doesn't yet exist in the PR-specific schema. So `is_incremental()` returns `false` and dbt runs a **full-refresh**. Two consequences: 1. Incremental models are often your largest tables → slow + expensive. 2. CI passes a full-refresh, but the actual merge into main would run *incrementally* — which can fail due to schema drift (especially with `on_schema_change='fail'`).

The fix

Zero-copy clone the relevant pre-existing incremental models into the PR schema **before** the slim build, so they exist and `is_incremental()` returns `true`.

The two-step CI job

```
        # Step 1: Clone modified incrementals (and downstream incrementals) that
already exist in prod
        dbt clone --select state:modified+,config.materialized:incremental,state:old \

                  --state path/to/prod

        # Step 2: Run Slim CI as normal
        dbt build --select state:modified+ --defer --state path/to/prod
```

Because step 1 created the incremental tables in the PR schema, step 2's `dbt build` runs them **incrementally** — matching what will happen post-merge in prod.

Prerequisites

- **dbt 1.6+** for `dbt clone`.
- **A warehouse that supports zero-copy clone** (Snowflake, BigQuery, Databricks). On unsupported warehouses, `dbt clone` falls back to pointer views — this trick doesn't help there.
- Slim CI configured to defer to a production environment with valid artifacts.

Why each selector

```
state:modified+,config.materialized:incremental,state:old
```

- `state:modified+` — the modified node and its descendants.
- `config.materialized:incremental` — restrict to incremental models only.
- `state:old` — only models that *already exist* in the prior state (i.e., production).

Why `state:old`?

Brand-new incremental models can't be cloned — they don't exist in prod yet. They should run in full-refresh mode in CI (matching what they'll do in prod on first merge). Without `state:old`, dbt would log warnings like "No relation found in state manifest for…" for those new models. With it, dbt skips them cleanly.

The schema-drift trade-off

If your model has `on_schema_change='fail'` and a PR adds a column: - **Incremental build** → fails (schema mismatch). - **Full-refresh build** → succeeds.

After this clone trick, your CI will now run **incrementally** and **fail** on the schema change. Two options:

Option	Trade-off
Let CI fail	You notice the schema change. You commit to running `dbt build --full-refresh --select my_incremental_model` in prod after merge. Blocks the PR until you decide.
Don't add the clone step	CI passes (full-refresh). Merge succeeds. Then the next scheduled prod incremental run breaks. Surprise.

Most teams prefer the **fail-loudly** option — known issues are easier than mysterious post-merge breakages.

What happens visually

Before the clone:

```
prod schema:    fct_orders (incremental, 100M rows)
PR schema:      (empty)

dbt build --select state:modified+
  → fct_orders is built fresh in PR schema
  → 100M rows rebuilt
  → is_incremental() = false everywhere
```

After the clone:

```
prod schema:    fct_orders (incremental, 100M rows)
PR schema:      fct_orders (zero-copy clone, ~free)

dbt build --select state:modified+
  → fct_orders found in PR schema
  → is_incremental() = true
  → only new rows processed
  → behaves exactly like the post-merge production run
```

Where this fits in the test-smarter framework

This is operational testing — making sure CI faithfully simulates production. It's not about data quality (that's tests/contracts) but about pipeline reliability (does the merge actually work?).

Key takeaways

- Without cloning, Slim CI accidentally runs incrementals as full-refreshes → slow, expensive, and *can* mask schema-drift failures.
- Add a `dbt clone --select state:modified+,config.materialized:incremental,state:old --state path/to/prod` step before `dbt build`.
- Requires dbt 1.6+ and a warehouse that supports zero-copy clone.
- `state:old` filters out brand-new incrementals that can't be cloned.
- Faithful CI = catching schema-drift failures before merge, not after.

Related

- `dbt clone` command reference.
- *To defer or to clone* blog (the conceptual framing).
- Slim CI / `state:modified+` / `--defer` / `--state`.
- `on_schema_change` incremental config.

Data Tests

Source: https://docs.getdbt.com/docs/build/data-tests **Type:** Documentation · Checkpoint 3

What they are

Data tests are SQL assertions about your data. They run as `select` queries; **failing rows returned = test fails**, zero rows = pass. Apply them to models, sources, seeds, and snapshots.

> Note: "data tests" is the new name for what dbt used to call "tests". The YAML key `tests:` is still accepted as an alias for `data_tests:`. The rename disambiguates from **unit tests**.

Two kinds

Generic (most common)

A parameterized `test` block. Reusable across many models/columns. dbt ships four: - `unique` - `not_null` - `accepted_values` - `relationships`

```
models:
  - name: orders
    columns:
      - name: order_id
        data_tests:
          - unique
          - not_null
      - name: status
        data_tests:
          - accepted_values:
              arguments:                    # v1.10.5+; older versions put values at top level
                values: ['placed', 'shipped', 'completed', 'returned']
      - name: customer_id
        data_tests:
          - relationships:
              arguments:
                to: ref('customers')
                field: id
```

Singular (one-off)

A `.sql` file in `tests/` that returns failing rows.

```
-- tests/assert_total_payment_amount_is_positive.sql
select
    order_id,
    sum(amount) as total_amount
from {{ ref('fct_payments') }}
group by 1
having total_amount < 0
```

- Test name = file name.
- ❌ Do **not** end with `;` — causes failure.
- ❌ Do **not** reference singular tests in `model_name.yml` — they're auto-discovered.
- ✅ You can add a description in a `tests/schema.yml`:

```
data_tests:
  - name: assert_total_payment_amount_is_positive
    description: "Refunds have a negative amount, so..."
```

How they execute

For each test, dbt compiles a `select` query that returns failing records. If the query returns 0 rows, the test passes.

`unique` test compiles to:

```
select * from (
    select order_id from analytics.orders where order_id is not null
    group by order_id having count(*) > 1
) validation_errors
```

`not_null` test compiles to:

```
select * from analytics.orders where order_id is null
```

Where to put tests (CP3 reminder)

- **`tests/`** — singular and generic test SQL.
- **`tests/generic/`** — custom generic test definitions.
- **`models/`** — unit test YAML lives here (NOT in `tests/`).
- Model-level data tests defined in `_<dir>__models.yml` next to the model.

Configs that matter

Config	Purpose
`severity: warn` / `error`	Warn vs fail the build
`warn_if` / `error_if` (`>0`, `>=N`)	Thresholds for severity flips
`tags`, `meta`	Selection and metadata
`store_failures`	Persist failing rows to a table for inspection
`store_failures_as`	Override the storage strategy
`where`	Filter rows the test runs over (e.g., partition pruning)
`limit`	Cap rows returned (rare)

Running only data tests

```
dbt test --select "test_type:data"
# v1.9+ Core also supports:
dbt test --resource-type test
```

Storing failures for inspection

```
dbt test --store-failures
```

- Saves failing rows to a table in a `dbt_test__audit` schema (or whatever you configure).
- Subsequent runs **replace** prior failures for the same test.
- Big productivity win during debugging — quickly query the failing rows.

Source tests

Same syntax, attached to source columns instead of model columns. See *Sources* doc and *Testing and documenting sources* (anchor in the sources page).

Severity patterns

- **error** — pipeline-failing concerns (customer-facing, financial, executive). From *Test smarter*: these are your top-priority data quality concerns.
- **warn** — nice-to-knows you'll act on within ~6 months.

More generic tests

Bring in packages: - `dbt_utils` — `not_constant`, `expression_is_true`, `mutually_exclusive_ranges`, … - `dbt-expectations` — Great Expectations-style assertions.

Key takeaways

- Data tests = SQL queries that should return zero failing rows.
- Two shapes: **generic** (reusable, parameterized) and **singular** (one-off SQL in `tests/`).
- Built-in generics: `unique`, `not_null`, `accepted_values`, `relationships`.
- `--store-failures` persists failing rows for inspection.
- The YAML key is now `data_tests:` (older `tests:` still works).
- `test_type:data` selector runs data tests but excludes unit tests.

Related

- Custom generic data tests; unit tests.
- Data test configurations (the full config reference).
- `dbt test` command, `test_type` selection.
- *Test smarter not harder* / *Where should tests go?*

Testing and Documenting Sources

Source: https://docs.getdbt.com/docs/build/sources#testing-and-documenting-sources **Type:** Documentation (section anchor) · Checkpoint 3

> ℹ️ This is a section of the larger *Sources* doc, called out separately in the certification study guide. The full sources cheat sheet lives at `checkpoint-1/08-sources.md`. This sheet zooms in on the testing/documenting subset.

Why this is a CP3 topic

"Tests live where the resource lives" — for sources, that means in the same YAML that declares the source. Source tests catch **fixable-at-source** issues (per *Test smarter*) and feed source freshness into your dependency graph.

The shape

Sources accept the same `description`, `data_tests`, and `columns` keys as models. Define them in `_<dir>__sources.yml` in the relevant staging folder.

```
# models/staging/jaffle_shop/_jaffle_shop__sources.yml
sources:
  - name: jaffle_shop
    description: "Replica of our app's Postgres DB"
    database: raw
    schema: jaffle_shop
    tables:
      - name: orders
        description: >
          One record per order. Includes cancelled and deleted orders.
        columns:
          - name: id
            description: Primary key of the orders table
            data_tests:
              - unique
              - not_null
          - name: status
            description: Note that the status can change over time
            data_tests:
              - accepted_values:
                  arguments:
                    values: ['placed','shipped','completed','returned']
```

What you can test on a source

The same toolkit as models: - Built-in generics: `unique`, `not_null`, `accepted_values`, `relationships`. - Custom generic tests from your project or packages (`dbt_utils`, `dbt-expectations`). - Singular tests in `tests/` referencing the source via `{{ source(...) }}`.

What source tests are *for* (per *Test smarter — where*)

Source tests should catch issues **fixable at the source system**. “Fixable” means: - You can fix it directly, OR - You know who can and have the relationship to actually get it done.

Examples of good source tests: - Duplicate records the source team can dedupe. - `not_null` on a field someone forgot to populate. - Primary-key uniqueness when duplicates are removable upstream.

What source tests should **not** carry: - “Cleanup work” you’re doing in staging (e.g., `not_null` on a column whose nulls you’ll filter in `stg_`). - Business-anomaly logic (that belongs at staging/intermediate/marts, not at the source level).

Documenting sources

Descriptions on the source, tables, and columns render in dbt’s docs site. Use them generously — sources are the seam between your warehouse and your dbt project, and downstream consumers often need to know “where does this come from and what does it mean.”

You can pull doc blocks in via `{{ doc('...') }}` for re-use.

How it relates to source freshness

Source-level config doesn’t only carry tests — it also carries `freshness` and `loaded_at_field`. Together they let `dbt source freshness` check whether the upstream load pipeline is on schedule.

Severity choice per *Test smarter — where*: - Source feeds **high-impact** outputs → freshness severity = **error**. - Source feeds nice-to-know outputs → severity = **warn**.

Selecting source tests

```
# Test only sources
dbt test --select "source:*"

# Test a specific source
dbt test --select "source:jaffle_shop"

# One source table
dbt test --select "source:jaffle_shop.orders"
```

Worth knowing

- A `relationships` test from a model column to a source field is perfectly valid.
- Source tests count toward overall test selection — including `test_type:data` and `--store-failures`.
- You cannot define unit tests on sources (unit tests are for SQL models only).

Key takeaways

- Source YAML supports `description`, `data_tests`, and `columns` — just like models.
- Tests on sources should target **fixable-at-source** data issues.
- Don’t double-test cleanup you’re handling in staging.
- Source freshness severity follows the high-impact rule (error vs warn).
- Select source tests via `source:` selectors.

Related

- Full Sources cheat sheet (CP1 doc list).
- Data tests / custom generic data tests.
- Source freshness (`dbt source freshness`).
- *Test smarter — where should tests go?*

Writing Custom Generic Data Tests

Source: https://docs.getdbt.com/best-practices/custom-generic-tests **Type:** Documentation · Checkpoint 3

What they are

Generic data tests beyond the four built-ins (`unique`, `not_null`, `accepted_values`, `relationships`). A reusable, parameterized assertion you define once and apply across many columns/models.

Where they live

Two valid locations: - ✅ **`tests/generic/`** — the conventional home for custom generic tests. - ✅ **`macros/`** — works because generic tests are macros under the hood. Use this when the test depends on other macros you’re co-defining.

The two standard arguments

Every generic test should accept one or both of: - **`model`** — the resource being tested, templated to its full relation name. Always called `model` even when applied to a source/seed/snapshot. - **`column_name`** — the column being tested. Only when the test operates at the column level.

Minimal example (column-level)

```
-- tests/generic/test_is_even.sql
{% test is_even(model, column_name) %}

with validation as (
    select {{ column_name }} as even_field
    from {{ model }}
),
validation_errors as (
    select even_field
    from validation
    where (even_field % 2) = 1
)
select * from validation_errors

{% endtest %}
```

Apply it:

```
# models/users.yml
models:
  - name: users
    columns:
      - name: favorite_number
        data_tests:
          - is_even:
              description: "This is a test"
```

Tests with extra arguments

```
-- tests/generic/test_relationships.sql (rebuild of the built-in)
{% test relationships(model, column_name, field, to) %}

with parent as (
    select {{ field }} as id from {{ to }}
),
child as (
    select {{ column_name }} as id from {{ model }}
)
select *
from child
where id is not null
  and id not in (select id from parent)

{% endtest %}
```

Apply with arguments in a dict (v1.10.5+ syntax; older versions place them at top level):

```
models:
  - name: people
    columns:
      - name: account_id
        data_tests:
          - relationships:
              description: "..."
              arguments:
                to: ref('accounts')
                field: id
```

`model` and `column_name` are provided by context — you don’t redeclare them.

Default config inside a generic test

You can `{{ config(...) }}` inside the test definition. Values become the *defaults* for every use of that test (overridable on each instance).

```
-- tests/generic/warn_if_odd.sql
{% test warn_if_odd(model, column_name) %}
    {{ config(severity = 'warn') }}
    select * from {{ model }} where ({{ column_name }} % 2) = 1
{% endtest %}
```

```
columns:
  - name: favorite_number
    data_tests:
      - warn_if_odd                    # severity: warn (default)
  - name: other_number
    data_tests:
      - warn_if_odd:
          arguments:
            severity: error            # overrides
```

Documenting custom generic tests

Document the underlying macro in a `schema.yml`:

```yaml
# macros/generic/schema.yml
macros:
  - name: test_not_empty_string             # the test block's name with `test_` prefix
    description: Complementary to not_null — also rejects empty strings.

    arguments:
      - name: model
        type: string
        description: Model Name
      - name: column_name
        type: string
        description: Column that should not be empty
```

> **Important:** the *macro* name is `test_<your_test_name>`. The *test block name* (the one users reference in YAML) omits the `test_` prefix.

Overriding a built-in

To replace a built-in test (e.g., make `unique` ignore nulls differently), define a test block with that exact name in your project. dbt will prefer the project's version over the built-in.

```sql
-- tests/generic/unique.sql
{% test unique(model, column_name) %}
    -- your custom logic
{% endtest %}
```

When to write one

- You write the same singular test pattern 3+ times → promote to a generic.
- You want the assertion to live as YAML metadata next to a column rather than as a `.sql` file.
- You need a default `severity`/`tags` policy across many models for a given check.

Don't reinvent

Many useful generic tests already exist: - `dbt_utils` — `not_constant`, `expression_is_true`, `mutually_exclusive_ranges`, `at_least_one`, `equal_rowcount`, … - `dbt-expectations` — Great-Expectations-flavored assertions, especially distribution and statistical checks.

Key takeaways

- Custom generics are `test` blocks defined in `tests/generic/` (or `macros/`).
- Accept `model` and/or `column_name` as the canonical arguments.
- Add extra arguments and pass them via `arguments:` in YAML (v1.10.5+).
- `{{ config(severity=...) }}` inside the test sets defaults that instances can override.
- Document the underlying macro using a `test_<name>` entry in `macros:`.
- Override built-ins by re-defining a block with the same name in your project.

Related

- Data tests (the consumer doc).
- Data test configurations (`severity`, `where`, `store_failures` …).
- `dbt_utils`, `dbt-expectations` packages.
- *Test smarter not harder* — what tests to write in the first place.

Data Test Configurations

Source: https://docs.getdbt.com/reference/data-test-configs **Type:** Documentation · Checkpoint 3

What it covers

The full set of configs you can apply to data tests, plus the precedence rules. Configs control severity, where the test runs, how failures are stored, and metadata (tags/meta/enabled).

Three places to set configs

Most specific wins.

1. **YAML properties** (next to where the test is applied — `models/.../*.yml`)
2. **`config()` block** in the test's SQL definition
3. **`dbt_project.yml`** under `data_tests:`

Precedence (specific instance of a generic test)

property YAML > generic test's SQL `config()` > `dbt_project.yml`

Precedence (singular test)

SQL `config()` in the singular test > `dbt_project.yml` (Singular tests **cannot** be configured from a properties YAML — only via their `config()` block or the project file.)

The test-specific configs

Config	Effect
`severity: error | warn`	Whether failure stops the build (`error`) or logs only (`warn`)
`error_if: "<expr>"`	Threshold to escalate to error (e.g., `>10`)
`warn_if: "<expr>"`	Threshold to warn
`fail_calc: "<sql expression>"`	How dbt counts failures (default: `count(*)`)
`limit: <int>`	Cap on failing rows the test returns
`store_failures: true | false`	Persist failing rows to a table
`store_failures_as: 'view' | 'table' | 'ephemeral'`	How to persist them
`where: "<sql>"`	Filter rows the test runs over (great for partition pruning)
`sql_header: "<string>"` (v1.12+)	Inject SQL before the test query (requires `require_sql_header_in_test_configs` flag)

The general configs

Config	Effect
`enabled: true | false`	Toggle the test
`tags: [...]`	Selection / grouping
`meta: {...}`	Free-form metadata
`database` / `schema` / `alias`	Where to write `store_failures` artifacts

Configuring in YAML (the common shape, v1.10.5+)

```yaml
models:
  - name: orders
    data_tests:
      - dbt_utils.equality:
          name: equality_fct_test_coverage
          description: ...
          arguments:
            compare_model: ref('compare_model')
          config:
            severity: warn
            warn_if: ">0"
            error_if: ">100"
            store_failures: true
            tags: ['nightly']
    columns:
      - name: order_id
        data_tests:
          - unique:
              config:
                severity: error
                where: "order_date >= dateadd('day', -30, current_date)"

          - not_null
```

Configuring in `dbt_project.yml`

```yaml
data_tests:
  my_project:
    +severity: warn                 # default for the whole project
    marts:
      +severity: error              # marts get strict
      +store_failures: true
    staging:
      +tags: ['hygiene']
```

Singular test config — only via SQL

```sql
-- tests/assert_revenue_positive.sql
{{ config(
    severity = 'error',
    warn_if = '>0',
    error_if = '>100',
    where = "transaction_date >= dateadd('day', -7, current_date)",
    store_failures = true
) }}

select transaction_id, revenue
from {{ ref('fct_transactions') }}
where revenue < 0
```

`store_failures` mechanics

- When enabled, dbt creates a table for failures (in `dbt_test__audit` schema by default).
- Failures **replace** prior failures for the same test.
- Schema/database/alias for the storage tables are configurable.

`where` — the cheap-test trick

For huge models, `where` lets you partition-prune in the test:

```yaml
- unique:
    config:
      where: "order_date >= dateadd('day', -30, current_date)"
```

Test runs only on the last 30 days instead of the whole table. Huge cost win.

`severity` / `warn_if` / `error_if`

```yaml
# Default behavior: warn if any failing rows
- not_null:
    config:
      severity: warn

# Custom thresholds
- some_test:
    config:
      severity: error    # base
      warn_if:  ">0"     # warn if >0 failures
      error_if: ">10"    # error if >10 failures
```

Tag examples

```yaml
- unique:
    config:
      tags: ['my_tag']         # selectable via --select tag:my_tag
```

Key takeaways

- Configure tests in three places; instance YAML > SQL `config()` > `dbt_project.yml`.
- Severity is the most-used config: `error`/`warn` with optional thresholds.
- `where` is the cost optimizer — limits test scope to relevant rows.
- `store_failures` persists failing rows for inspection (in `dbt_test__audit` by default).
- Singular tests configure via SQL `config()` only (no property YAML).
- v1.12+ `sql_header` is gated by the `require_sql_header_in_test_configs` flag.

Related

- Data tests, custom generic tests.
- `severity`, `store_failures`, `where`, `fail_calc`, `limit` resource configs.
- `--store-failures` CLI flag.
- *Test smarter not harder* — when to use which severity.

Unit Tests

Source: https://docs.getdbt.com/docs/build/unit-tests **Type:** Documentation · Checkpoint 3

What they are

Tests that validate **transformation logic** against **synthetic** inputs **before** the model is materialized. dbt feeds you mocked input rows, runs the model's SQL, and asserts the output matches expectations. Failure stops `dbt build` before any warehouse cost is incurred.

Introduced in dbt v1.8. Distinct from **data tests** (which run against real, built data).

When to add a unit test

- **Complex SQL logic** — regex, date math, window functions, CASE-WHENs with many branches, truncation.
- **Custom functions** — your model's logic looks function-like.
- **Edge cases not in real data yet** — protect against them.
- **Bug-replay** — encode past bugs as tests.
- **Before a refactor** — pin behavior so you notice unintended changes.
- **High-criticality models** — public, contracted, or directly upstream of an exposure.

Don't bother

- Trivial `min`/`max`/`sum` calls — the warehouse already tests those.
- Models that are just renames/casts (covered by data tests).

When NOT to run unit tests

Don't run in production — the inputs are static, you're spending compute for nothing. Run in dev and CI only.

Exclude in prod jobs:

```bash
# v1.10 and earlier
DBT_EXCLUDE_RESOURCE_TYPES=unit_test dbt build
# v1.11+
DBT_ENGINE_EXCLUDE_RESOURCE_TYPES=unit_test dbt build
# Or CLI flag:
dbt build --exclude-resource-type unit_test
```

Run only unit tests:

```bash
dbt test --select test_type:unit
```

Where they live

- Unit tests **must** live under `model-paths` (i.e. `models/`), not in `tests/`.
- Define under `unit_tests:` in a properties YAML next to the model.
- Fixture files (CSV, SQL) live in a `fixtures/` subdir of any test path (e.g., `tests/fixtures/`).

Basic example

Model `dim_customers.sql` calculates `is_valid_email_address`. Unit test it:

```yaml
unit_tests:
  - name: test_is_valid_email_address
    description: "Check is_valid_email_address captures known edge cases."

    model: dim_customers
    given:
      - input: ref('stg_customers')
        format: dict
        rows:
          - {email: cool@example.com,    email_top_level_domain: example.com}

          - {email: cool@unknown.com,    email_top_level_domain: unknown.com}

          - {email: badgmail.com,        email_top_level_domain: gmail.com}

          - {email: missingdot@gmailcom, email_top_level_domain: gmail.com}

      - input: ref('top_level_email_domains')
        format: dict
        rows:
          - {tld: example.com}
          - {tld: gmail.com}
    expect:
      format: dict
      rows:
        - {email: cool@example.com,    is_valid_email_address: true}
        - {email: cool@unknown.com,    is_valid_email_address: false}
        - {email: badgmail.com,        is_valid_email_address: false}
        - {email: missingdot@gmailcom, is_valid_email_address: false}
```

You only need to specify the columns your test actually cares about. dbt fills in the rest with NULL/default.

Three fixture formats

Format	When to use
`dict` (inline)	Short, readable, common case
`csv` (inline or file)	Wider fixtures, easier to scan; file form for reusability
`sql` (inline or file)	When you need warehouse-specific types (BigQuery STRUCT, JSON)

CSV / SQL via file:

```yaml
- input: ref('top_level_email_domains')
  format: csv
  fixture: my_csv_fixture_file       # tests/fixtures/my_csv_fixture_file.csv
```

Prerequisites for the parents

The *direct parents* of the model under test must **exist in the warehouse** before you can run the unit test. To save cost:

```bash
dbt run --select "stg_customers top_level_email_domains" --empty
```

Or just `dbt build` — it processes unit tests *before* materializing, then data tests after.

How dbt processes them with `dbt build`

1. Run unit tests against the SQL model.
2. If they pass, materialize the model.
3. Run data tests on the materialized model.

If unit tests fail, the model is not built → no warehouse spend.

Running unit tests

```bash
dbt test --select dim_customers                     # all tests on the model
dbt test --select "dim_customers,test_type:unit"    # unit tests on this model
dbt test --select test_is_valid_email_address       # one specific unit test
dbt test --select test_type:unit                    # all unit tests
```

Diff output on failure

On failure dbt shows a row-level diff:

```
actual differs from expected:
@@ ,email           ,is_valid_email_address
→  ,cool@example.com,True→False
```

The arrow shows the column where actual ≠ expected.

Unit testing incremental models

- You can **override macros, vars, or env vars** in the unit test config.
- This lets you test the incremental model in both **full-refresh** and **incremental** modes.
- **Expected output = the result of the materialization** (what gets merged/inserted), not the final table state.
- The incremental model must exist in the warehouse before running. Use `dbt run --select config.materialized:incremental --empty` to bootstrap empty tables cheaply.

Adapter caveats

- **BigQuery**: must specify all fields of a STRUCT in fixtures — no partial STRUCTs.
- **Redshift**: known limitations; workaround required. Sources must be in the same database as the models.

Key takeaways

- Unit tests validate transformation **logic** on **synthetic inputs**, before materialization.
- v1.8+ feature. Live in YAML under `unit_tests:`, in `models/`.
- Three fixture formats: `dict`, `csv`, `sql`. Specify only the columns you care about.
- Run in dev and CI; **exclude** in production with `--exclude-resource-type unit_test`.
- `dbt build` runs unit tests → materializes → data tests, all in order.
- For incrementals, expected output is the merge/insert result, not the final table.

Related

- Data tests (the post-build counterpart).
- `dbt test`, `test_type:unit` selector.
- `--empty` flag (cheap parent bootstrap).
- Unit test properties reference.

`dbt test` (CP3 angle)

Source: https://docs.getdbt.com/reference/commands/test **Type:** Documentation · Checkpoint 3 (also covered in CP1)

> ℹ️ Full command reference cheat sheet lives at `checkpoint-1/15-cmd-test.md`. This sheet emphasizes what CP3 cares about: type filtering, indirect selection, and where `dbt test` fits in the resilience story.

Recap of what `dbt test` does

Runs **data tests** (generic + singular) AND **unit tests** declared on SQL models. Doesn't build models — expects them to already exist (or use `dbt build` which orchestrates both).

CP3 essentials

Filter by test type

```
dbt test                                        # everything
dbt test --select test_type:data                # only data tests
dbt test --select test_type:unit                # only unit tests
dbt test --select test_type:generic             # only generic data tests

dbt test --select test_type:singular            # only singular data tests
```

`test_type` selection works on both Core and Fusion. v1.9+ Core also supports `dbt test --resource-type test` (filters to *all* test types) and `--resource-type unit_test`.

Indirect selection

`dbt test --select my_model` pulls in: - Tests defined directly on the model's columns/properties. - Generic tests referencing the model (e.g., a `relationships` test pointing at it). - Tests are excluded if their other parents aren't selected.

This is the same indirect-selection rule used by `dbt build` — selecting the model gives you its tests for free.

Combine selectors

```
# Unit tests on one model
dbt test --select "dim_customers,test_type:unit"

# Failed tests from last run
dbt test --select result:fail --state path/to/prod

# Slim CI: only test modified things
dbt test --select state:modified --defer --state path/to/prod
```

Result-based selection (CP3 favorite)

Pair `result:` selectors with `--state` to rerun failures cheaply.

```
dbt test --select result:fail --state path/to/prod
dbt test --select result:fail --exclude my_known_flaky_test --state path/to/
prod
```

⚠️ **Gotcha**: `dbt test` writes a new `run_results.json` over the previous one. So `result:fail` + `result:error` won't both work in a sequence of separate `dbt run` + `dbt test` invocations — `dbt test` clobbers the earlier results. To compose them, use `dbt build` instead (it writes one combined results file).

Where this fits in resilience

CP3's testing story is: 1. Write the right tests (Test smarter not harder). 2. Put them in the right places (Where should tests go?). 3. Run them with discipline: - In dev: `dbt test --select my_changes+` - In CI: `dbt build --select state:modified+ --defer --state path/to/prod` - On failure: `dbt test --select result:fail --state path/to/prod`

The command is the same; the *selection* is what makes the workflow resilient and cost-aware.

Storing failures for inspection

```
dbt test --store-failures
```

Failures get persisted to a `dbt_test__audit` schema table (or wherever you configure). Subsequent runs replace prior failures for the same test. See *Data test configurations* for `store_failures` / `store_failures_as` configs.

When `dbt build` is the better choice

- Production runs — `dbt build` runs unit tests before materializing models and data tests after, in DAG order, with failing tests skipping downstream.
- Combining run + test result selection (`result:error+ result:fail+`) — only works inside one `dbt build` invocation.

`dbt test` standalone fits when: - You're iterating on tests themselves (not models). - Tests-only CI jobs. - Replaying a failed test set from a prior run.

Examples (CP3 patterns)

```
# Source-freshness-driven test
dbt source freshness
dbt test --select source_status:fresher+ --state path/to/prod

# Test only what an exposure depends on
dbt test --select +exposure:weekly_kpis

# Slim CI test only
dbt test --select state:modified --defer --state path/to/prod
```

Key takeaways

- `dbt test` runs both data tests and unit tests; filter via `test_type:`.
- Indirect selection: pick the model, get the tests.
- `result:fail` / `result:error` rerun failed tests cheaply (with `--state`).
- `dbt test` overwrites `run_results.json` — for combined run+test result selection use `dbt build`.
- For production, prefer `dbt build` so test failures skip downstream models.

Related

- Full `dbt test` reference in CP1 cheat sheet.
- Data tests, unit tests, custom generic tests, data test configs.
- Slim CI / `--defer` / `state:` / `result:` selectors.
- `dbt build` for integrated DAG execution.

Exposures

Source: https://docs.getdbt.com/docs/build/exposures **Type:** Documentation · Checkpoint 3

What it is

An exposure declares a **downstream use** of your dbt project — a dashboard, ML model, application, notebook, or analysis. By naming it, you bring downstream consumers into the dbt DAG: you can `select` them, run/test their upstream dependencies, and surface them in the docs site.

Two ways exposures get created

- **Manual** — you write YAML.
- **Automatic** (dbt Cloud / supported integrations) — Cloud creates downstream exposures based on connected BI tools (Tableau, Looker, etc.). They behave like manual exposures but don't have a YAML file.

Manual declaration

```yaml
# models/marts/finance/_finance__exposures.yml
exposures:
  - name: weekly_jaffle_metrics       # required, snake_case
    label: Jaffles by the Week        # optional, display name
    type: dashboard                   # required: dashboard|notebook|analysis|ml|application
    maturity: high                    # optional: high|medium|low
    url: https://bi.tool/dashboards/1 # optional; activates "View this exposure" link
    description: >
      Did someone say "exponential growth"?

    depends_on:                       # expected
      - ref('fct_orders')
      - ref('dim_customers')
      - source('gsheets', 'goals')
      - metric('count_orders')

    owner:                            # required: name OR email
      name: Callum McData
      email: data@jaffleshop.com
```

Required vs optional

Required: - `name` — snake_case, unique. No spaces or special chars (use `label` for that). - `type` — one of `dashboard`, `notebook`, `analysis`, `ml`, `application`. - `owner` — `name` or `email` required; additional fields allowed.

Expected: - `depends_on` — list of `ref()`, `source()`, `metric()`. (Almost never a raw `source()` — exposures usually depend on marts.)

Optional: - `label`, `url`, `maturity`, `description`, `tags`, `meta`, `enabled`.

`depends_on` ≠ SQL `-- depends_on`

- **Exposure YAML `depends_on:`** — declares dependencies for the DAG.
- **SQL `-- depends_on:`** comment directive — used inside model SQL files to add explicit dependencies. Different feature, don't confuse.

What exposures unlock

Selection

```bash
# Build all upstream of an exposure
dbt run --select +exposure:weekly_jaffle_metrics

# Test all upstream
dbt test --select +exposure:weekly_jaffle_metrics

# Find sources upstream of all exposures
dbt ls --select "+exposure:*" --resource-type source
```

The `exposure:` selector method combined with the `+` graph operator → critical for "build everything my dashboard needs" workflows.

Documentation

- Each exposure gets a dedicated page in the docs site (and dbt Explorer/Catalog).
- The `url` field activates the "View this exposure" link in docs.
- Marked with an orange **EXP** indicator in the DAG visualization.

Data health tiles

Cloud generates **data health tiles** for exposures — at-a-glance freshness/test status that you can embed in BI tools.

Maturity values

- `high` — well-established, widely trusted (executive dashboards).
- `medium` — useful but not yet broadly relied on.
- `low` — new/experimental.

Used purely for organization/communication; doesn't change behavior.

Where exposures should live (per CP3 structure guide)

Generally next to the marts they consume:

```
models/marts/finance/
├── _finance__models.yml
├── _finance__exposures.yml
├── orders.sql
└── payments.sql
```

Operational patterns

```bash
# Build all upstreams of a critical dashboard before a release
dbt build --select +exposure:executive_summary

# Pre-deploy sanity: test everything any exposure depends on
dbt test --select +exposure:*

# What sources feed any exposure?
dbt ls --select "+exposure:*" --resource-type source
```

Project-level enabled toggle

```yaml
# dbt_project.yml
exposures:
  +enabled: true              # or false to disable all exposures
```

Key takeaways

- Exposures = downstream consumers (dashboards, ML, apps) declared in YAML.
- Five `type` values, `owner` required, `depends_on` lists upstream refs/sources/metrics.
- The `exposure:` selector + `+` operator builds/tests everything an exposure relies on.
- Render in docs site / Catalog; pair with health tiles for at-a-glance status.
- Live alongside the marts they consume, in `_<area>__exposures.yml`.

Related

- Exposure properties (full reference / project-level config).
- `exposure:` node selection method.
- Data health tiles.
- *Test smarter — where should tests go?* (exposures inform test prioritization).

Exposure Properties

Source: https://docs.getdbt.com/reference/exposure-properties **Type:** Documentation · Checkpoint 3

What it is

The full reference for the YAML shape of an exposure. The build doc (`exposures.md`) shows the common case; this is the complete property list.

Where they live

Inside a properties YAML under `models/` (any depth). You can put `exposures:` in the same YAML file as `sources:` and `models:`, or in a dedicated `_<area>__exposures.yml`. File name is up to you; the file just needs to be under `model-paths`.

Full schema

```yaml
exposures:
  - name: <snake_case_name>             # required — only letters, numbers, underscores
    description: <markdown_string>      # optional
    type: dashboard | notebook | analysis | ml | application    # required

    url: <string>                       # optional — activates "View this exposure" link
    maturity: high | medium | low       # optional — communicates stability

    enabled: true | false               # optional (also configurable in dbt_project.yml)
    config:                             # v1.10: tags and meta moved under config
      tags: ['nightly', 'critical']
      meta: {owner_team: 'finance', tier: 1}
      enabled: true | false             # alternative location for enabled

    owner:                              # required — at least 'name' or 'email'
      name: <string>
      email: <string>
    depends_on:                         # expected — list of refs/sources/metrics
      - ref('model')
      - ref('seed')
      - source('source_name', 'table_name')
      - metric('metric_name')
    label: <human-friendly name>        # optional — title-cased / with spaces
```

Name vs label

- **`name`** — machine identifier. snake_case only. Used in CLI selectors (`--select exposure:my_name`).
- **`label`** — display string for the docs site. Can have spaces, capitals, special characters.

Worked examples (different types)

```yaml
exposures:
  - name: weekly_jaffle_metrics
    label: Jaffles by the Week
    type: dashboard
    maturity: high
    url: https://bi.tool/dashboards/1
    description: "Weekly KPIs for jaffle sales."
    depends_on:
      - ref('fct_orders')
      - ref('dim_customers')
      - source('gsheets', 'goals')
      - metric('count_orders')
    owner:
      name: Callum McData
      email: data@jaffleshop.com

  - name: jaffle_recommender
    type: ml
    maturity: medium
    url: https://jupyter.org/mycoolalg
    description: "Discover Sandwiches Weekly recommender."
    depends_on:
      - ref('fct_orders')
    owner:
      name: Data Science Drew
      email: data@jaffleshop.com

  - name: jaffle_wrapped
    type: application
    description: "End-of-year favorites recap for users."
    depends_on: [ ref('fct_orders') ]
    owner: { email: summer-intern@jaffleshop.com }
```

Project-level config

Currently only `enabled` is supported at the project level:

```yaml
# dbt_project.yml
exposures:
  +enabled: true
```

You can scope it to a directory if you organize exposure YAMLs into subfolders:

```yaml
exposures:
  +enabled: true
  experimental:
    +enabled: "{{ target.name == 'dev' }}"
```

v1.10 config-vs-property change

Pre-v1.10, `tags` and `meta` were top-level keys. In v1.10+ they live under `config:`:

```yaml
exposures:
  - name: ...
    config:
      tags: ['critical']
      meta: {tier: 1}
```

Set `enabled` either at top level or inside `config:` — both supported.

Where the URL leads

When `url` is set, the rendered docs page gets a "View this exposure" link to that URL — typically the dashboard or notebook the exposure represents. Use it; reviewers and consumers click it.

`description` rendering

Markdown is supported. Use it to document: - The audience. - What questions the exposure answers. - Refresh cadence and SLA. - Known caveats / limitations.

For complex descriptions, use doc blocks:

```yaml
description: "{{ doc('exposure_weekly_jaffle_metrics') }}"
```

With the doc block in a `_<dir>__docs.md`:

```
{% docs exposure_weekly_jaffle_metrics %}
## Weekly Jaffle Metrics
This dashboard tracks weekly KPIs ...
{% enddocs %}
```

What you cannot put on an exposure

- `columns` — exposures have no columns; they're not tables.
- `data_tests` — exposures aren't tested directly. Test the resources they depend on.
- `materialized` — exposures aren't materialized; they're references.

Key takeaways

- Required: `name` (snake_case), `type` (5 options), `owner` (with name or email).
- Expected: `depends_on` (refs/sources/metrics).
- Optional: `label`, `url`, `maturity`, `description`, plus `config: {tags, meta, enabled}`.
- v1.10+: `tags`/`meta` moved under `config:`.
- Project-level `+enabled` for batch toggling.
- Use `label` for display, `name` for machine identifier; doc blocks for rich descriptions.

Related

- Exposures (the build doc).
- `exposure:` node selection method.
- Doc blocks for description re-use.
- Data health tiles (Cloud feature on top of exposures).

`dbt source freshness`

Source: https://docs.getdbt.com/reference/commands/source#dbt-source-freshness
Type: Documentation · Checkpoint 3

What it does

`dbt source` provides one subcommand: `dbt source freshness`. It queries each configured source table to determine how stale its data is, comparing against your declared `warn_after` / `error_after` thresholds.

- Stale beyond `warn_after` → **warn** (exits 0 by default).
- Stale beyond `error_after` → **error** (nonzero exit).
- Healthy → **pass**.

Prerequisite: configure freshness on sources

```yaml
sources:
  - name: jaffle_shop
    database: raw
    config:                          # v1.9+ structure
      freshness:
        warn_after: {count: 12, period: hour}
        error_after: {count: 24, period: hour}
      loaded_at_field: _etl_loaded_at    # v1.10+ inside config
    tables:
      - name: customers
      - name: orders
        config:
          freshness:                     # stricter override
            warn_after: {count: 6, period: hour}
            error_after: {count: 12, period: hour}
            filter: "datediff('day', _etl_loaded_at, current_timestamp) < 2"

      - name: product_skus
        config:
          freshness: null                # opt out
```

Key fields: - `warn_after` / `error_after` — thresholds; at least one required for freshness to compute. - `loaded_at_field` — column to use as "last loaded at." Required unless dbt can read warehouse metadata. - `filter` — `WHERE` clause for the freshness query (huge perf win on large tables).

Source-level config cascades to all child tables; tables can override.

Common commands

```
# Check all sources
dbt source freshness

# Specific source
dbt source freshness --select "source:snowplow"

# Specific table within a source
dbt source freshness --select "source:snowplow.event"

# Custom output path
dbt source freshness --output target/source_freshness.json
```

The `sources.json` artifact

`dbt source freshness` writes `target/sources.json` — used by downstream tools (Cloud dashboards, custom alerting, `source_status:fresher+` selection).

Example:

```
{
  "sources": {
    "source.project.jaffle_shop.orders": {
      "max_loaded_at": "2024-...",
      "snapshotted_at": "2024-...",
      "max_loaded_at_time_ago_in_s": 481.3,
      "state": "pass",
      "criteria": { "warn_after": {...}, "error_after": {...} }
    }
  }
}
```

Override the destination with `-o` / `--output`.

How dbt computes freshness

For each table, dbt runs:

```
select max({{ loaded_at_field }}) as max_loaded_at,
       convert_timezone('UTC', current_timestamp()) as calculated_at
from {{ source_relation }}
{{ optional filter clause }}
```

Compares `current_timestamp - max_loaded_at` against your thresholds.

The `source_status:fresher+` selector

After two `dbt source freshness` runs (one prior, one current), dbt can compare to see which sources are **fresher** than last time. Combine with `--state` to rebuild only downstream of refreshed sources:

```
# Initial run produces sources.json
dbt source freshness

# Later: rebuild only downstream of sources that have new data
dbt source freshness
dbt build --select source_status:fresher+ --state path/to/prod
```

Pairs perfectly with Slim CI to drive efficient incremental orchestration.

Severity choice (per *Test smarter — where*)

- Source feeds **high-impact** outputs (customer-facing/financial/executive) → `error_after` defined → stale data fails the job.
- Source feeds nice-to-knows → use `warn_after` only, no `error_after`. Pipeline keeps running; you see the warning.

State-aware orchestration (Fusion)

Fusion's state-aware orchestration auto-tracks source freshness via warehouse metadata. You only need explicit freshness config when you want: - SLA alerting. - Custom freshness logic via `loaded_at_field` / `loaded_at_query` (streaming, partial loads). - Freshness for source **views** (Fusion can't determine freshness from view metadata — treats as always fresh).

Operational patterns

```
# Snapshot freshness on a 30-min cron
*/30 * * * *  dbt source freshness

# Hourly: rebuild only what changed
0 * * * *  dbt build --select source_status:fresher+ --state path/to/
prod

# Alert on staleness in Cloud
# - Configure under Execution settings → freshness snapshot
```

Key takeaways

- `dbt source freshness` checks source staleness against `warn_after` / `error_after`.
- Needs `loaded_at_field` (or warehouse metadata, or `loaded_at_query`) and at least one threshold.
- Writes `target/sources.json` — fuel for `source_status:fresher+` builds.
- `filter` is your performance lever on huge source tables.
- Severity choice maps to downstream impact: error for critical, warn for nice-to-know.
- Fusion + state-aware orchestration automates freshness via warehouse metadata.

Related

- Sources doc (declaration mechanics).
- Source configs (the freshness keys in detail).
- `source_status:fresher+` selector.
- `dbt source` command parent doc.

About State in dbt

Source: https://docs.getdbt.com/reference/node-selection/state-selection **Type:** Documentation · Checkpoint 3

The mental model

dbt is **stateless** and **idempotent**: same code + same raw data → same result, no matter how many times you run.

But dbt **stores** state — point-in-time snapshots of project resources, database objects, and invocation results — as **artifacts** in `target/`. State-aware features use those artifacts to *inform* operations without changing the stateless guarantee.

The artifacts that constitute state

- **`manifest.json`** — parsed project state (nodes, configs, refs, sources, tests).
- **`run_results.json`** — outcomes of the last invocation (pass/fail/error/warn per node).
- **`sources.json`** — output of `dbt source freshness`.
- **`catalog.json`** — warehouse metadata (built by `dbt docs generate`).

The `--state` flag

Pass the path to a previous run's `target/` (or just the artifacts):

```
dbt build --select state:modified+ --state path/to/prod
```

Now dbt can compare the current project against the saved state — and selectors like `state:modified` become meaningful.

What state unlocks (three big features)

1. The `state` selector method

Selects resources that are new or modified by comparing current project to the state manifest.

```
dbt run  --select state:modified+ --state path/to/prod
dbt test --select state:modified+ --state path/to/prod
```

2. Deferral (`--defer`)

For models not selected in the current run, dbt overrides `ref()` to point at the production schema's pre-built objects. No new objects created.

```
dbt build --select state:modified+ --defer --state path/to/prod
```

This is **Slim CI** in one line.

3. `dbt clone`

Zero-copy clones nodes based on their location in the state manifest.

```
dbt clone --select state:modified+,config.materialized:incremental,state:old \
          --state path/to/prod
```

Other state-driven selectors (preview from *Methods* doc)

Selector	Selects
`state:new`	Resources that don't exist in the state manifest
`state:modified`	Resources whose code/config has changed
`state:modified.body`	Only models with changed SQL body
`state:modified.configs`	Only changed configs
`state:modified.persisted_descriptions`	Changed `description` text
`state:modified.relation`	Changed materialization or alias
`state:modified.macros`	Changed macros (incl. indirect)
`state:modified.contract`	Contract changes
`state:old`	Exists in state, used to filter to "already-existing"
`state:unmodified`	The opposite of modified
`result:pass` / `fail` / `error` / `warn` / `skipped`	From `run_results.json` of prior run
`source_status:fresher+`	Sources fresher than the prior `sources.json`

Slim CI — the canonical use case

```
dbt build --select state:modified+ --defer --state path/to/prod
```

- Compares CI branch to production manifest.
- Builds only the modified models and their descendants.
- Reads unchanged parents from the production schema via defer.
- Runs tests on the modified subset.

Result: cheaper, faster CI that mimics post-merge behavior.

Source-freshness-driven builds

```
dbt source freshness
dbt build --select source_status:fresher+ --state path/to/prod
```

Only rebuild downstream of sources whose data actually changed.

Composing state with other selectors

```
# Rerun failed models AND newly modified models, plus their downstream
dbt build --select state:modified+ result:error+ \
          --defer --state path/to/prod

# Rerun failed tests but exclude known-flaky ones
dbt test --select result:fail --exclude my_flaky_test \
         --defer --state path/to/prod
```

Caveats

See *State comparison caveats* for the full list. The headline ones: - **`state:modified` can have false positives** when configs are environment-dependent (different schemas in dev vs prod). The behavior-change flag `state_modified_compare_more_unrendered_values: true` fixes this by comparing unrendered configs. - **State must be from the right environment.** Using a CI state in production (or vice versa) gives nonsensical results. - **`dbt test` overwrites**

`run_results.json` — separate run + test calls compose poorly; use `dbt build` to combine.

How to obtain state

- **Production run output** — copy `target/manifest.json` and `target/run_results.json` from your prod job to a known location.
- **dbt Cloud** — Cloud manages state for you automatically when you use Slim CI / "defer to production."
- **`--defer-state` / `defer_state`** — separate flag for *just* defer state, in case you want defer state from one place and `--state` from another.

Key takeaways

- dbt is stateless; state is *informational*, used by `--state` to enable smarter selection.
- Three big consumers: `state:` selectors, `--defer`, `dbt clone`.
- Slim CI is the killer pattern: `state:modified+ --defer --state path/to/prod`.
- `result:` and `source_status:fresher` selectors compose with state to drive cost-aware reruns.
- Use the right state for the environment; watch for `state:modified` false positives.

Related

- *Node selector methods* — the `state:`, `result:`, `source_status:` reference.
- `dbt clone`, `dbt retry`.
- Slim CI / Best practice workflows.
- `state_modified_compare_more_unrendered_values` behavior flag.

Node Selector — `state:` Method (and friends)

Source: https://docs.getdbt.com/reference/node-selection/methods#state **Type:** Documentation · Checkpoint 3

What it is

A reference for the `state:` selector method (and the closely related `result:`, `source_status:`, `exposure:`, `config:`, etc.). These are how you turn dbt artifacts into selection power.

Selector syntax (recap)

```
--select method:value
```

Method is optional in many cases (`--select my_model` infers `fqn`/`file`/`path`). Wildcards: `*`, `?`, `[abc]`, `[a-z]`.

The `state:` selectors

Selector	Meaning
`state:new`	Resources missing from the state manifest (newly added in current code)
`state:modified`	Resources changed since the state was captured (code, config, etc.)
`state:modified.body`	Just the SQL body of models changed
`state:modified.configs`	Configs changed
`state:modified.persisted_descriptions`	Descriptions in YAML changed
`state:modified.relation`	Materialization or alias changed
`state:modified.macros`	Macros (direct or indirect) changed
`state:modified.contract`	Model contract changed
`state:old`	Resource exists in state — used to filter to already-existing things
`state:unmodified`	Opposite of modified

Combine with `+` graph operators to expand to downstream/upstream:

```
dbt build --select state:modified+ --defer --state path/to/prod
```

All `state:` selectors **require** the `--state` flag pointing at a directory with `manifest.json`.

The `result:` selectors

Use the previous invocation's `run_results.json`.

Selector	Selects
`result:pass`	Nodes that passed
`result:warn`	Warned
`result:fail`	Failed (test-specific)
`result:error`	Errored (any node type)
`result:skipped`	Skipped (e.g., due to upstream failure)

Note: - `result:fail` is specific to tests. Tests don't have downstream nodes, so `result:fail+` is the same as `result:fail`. Use `1+result:fail` to also include the model the test was on. - `result:error` selects any node that errored (model, test, snapshot, …).

Examples:

```
# Rerun errored models from last run
dbt run --select result:error --state path/to/artifacts

# Rerun failed tests AND their parent model(s)
dbt build --select 1+result:fail --state path/to/artifacts

# Rerun errored models + their downstream
dbt build --select result:error+ result:fail+ --state path/to/artifacts

# Rerun failed tests excluding known flaky
dbt test --select result:fail --exclude my_flaky_test --state path/to/artifacts
```

⚠️ `dbt test` overwrites `run_results.json` — combining `result:error` + `result:fail` only works inside a single `dbt build` invocation.

The `source_status:` selector

Uses two `sources.json` files (state vs current) to find sources whose data changed.

```
dbt source freshness                                          # snapshot current
dbt build --select source_status:fresher+ --state path/to/prod  # rebuild downstream
```

- Requires a current `dbt source freshness` run *and* a `--state` pointing at a prior one.
- The `+` graph operator typically follows — you want what's downstream of the refreshed sources.

Other useful methods (CP3-relevant)

`exposure`

Selects parents of an exposure. Always use with `+`.

```
dbt build --select +exposure:executive_summary
dbt ls    --select +exposure:* --resource-type source
```

`config`

Match arbitrary configs.

```
dbt run --select config.materialized:incremental
dbt run --select config.tags:nightly
dbt run --select config.meta.contains_pii:true
```

Used in CP3 for the clone-incremental pattern:

```
state:modified+,config.materialized:incremental,state:old
```

`resource_type`

Filter by node type.

```
dbt list --select resource_type:test
dbt build --select resource_type:exposure        # everything upstream of exposures
dbt build --exclude-resource-type unit_test      # production exclude pattern
```

`tag`

```
dbt run --select tag:nightly
```

`package`

```
dbt run --select package:snowplow
dbt run --select package:this      # current project only
```

`path` / `file` / `fqn`

Path-based selection.

```
dbt run --select path:models/staging/github
dbt build --select file:my_function.sql
dbt run --select fqn:my_project.staging.stg_orders
```

`version`

For versioned models:

```
selectors:
  - name: exclude_old_versions
    default: "{{ target.name == 'dev' }}"
    definition:
      method: fqn
      value: "*"
      exclude:
        - method: version
          value: old
```

Values: `latest`, `prerelease`, `old`, or a specific version number.

Set operators

- **Space-separated** = **union**: `--select tag:a tag:b`.
- **Comma-separated** = **intersection**: `--select tag:a,tag:b`.
- `+`, `-` graph operators expand downstream/upstream.

Examples in context

```
# Slim CI
dbt build --select state:modified+ --defer --state path/to/prod

# Clone incremental models before Slim CI build
dbt clone --select state:modified+,config.materialized:incremental,state:old \
          --state path/to/prod

# Reprocess errored models AND failed tests in one invocation
dbt build --select state:modified+ result:error+ result:fail+ \
          --defer --state path/to/prod

# Build everything an executive dashboard depends on
dbt build --select +exposure:executive_summary
```

Key takeaways

- `state:` selectors require `--state` and turn artifact diffing into selection.
- `result:` reruns failed/errored nodes from the prior invocation.
- `source_status:fresher+` reruns downstream of refreshed sources.
- `exposure:`, `config:`, `tag:`, `resource_type:`, `version:` round out the toolbox.
- Space = union, comma = intersection, `+` = graph expansion.
- The clone-incremental pattern is the canonical chained selector: `state:modified+,config.materialized:incremental,state:old`.

Related

- *About state in dbt* (the artifacts that make this possible).
- *State comparison caveats* (false-positive gotchas).
- Graph operators / set operators reference.
- YAML selectors (named, reusable selection sets).

`dbt retry`

Source: https://docs.getdbt.com/reference/commands/retry **Type:** Documentation · Checkpoint 3

What it does

Re-executes the **last invocation from the point of failure**. dbt reads `run_results.json` (in `target/` by default, or wherever `--state` points), skips everything that already succeeded, and reruns the failing nodes (and any that were skipped because of them).

If the previous command finished successfully, `dbt retry` is a no-op.

Supported commands

`build`, `compile`, `clone`, `docs generate`, `seed`, `snapshot`, `test`, `run`, `run-operation`.

What it inherits from the prior invocation

- The original `--select` / `--exclude` / `--selector` choices.
- The same target, profile, vars (unless you override).

In **Core**, you cannot override the selection — retry reuses the prior run's selection set exactly.

In **Fusion**, you **can** narrow the retry scope with `--select`, `--exclude`, `--selector` — those override the prior invocation's selection.

Core flags

```
--threads INT             override original thread count
--vars YAML               override vars
--target NAME             different target
--profile NAME            different profile
--profiles-dir PATH       where profiles.yml lives
--project-dir PATH        where dbt_project.yml lives
--target-path PATH        override target directory
--state PATH              read run_results.json from elsewhere (defaults to
target/)
--full-refresh            force incrementals to full-refresh
```

`dbt retry --help` for the full list your local build supports.

Fusion flags (v2.0+)

```
-t, --target TARGET
--project-dir PATH
--profile PROFILE
--profiles-dir PATH
--packages-install-path PATH
--target-path PATH
--vars VARS
--select / --exclude / --selector       (Fusion only — overrides prior
selection)
```

dbt Platform CLI note

When using the dbt Platform CLI against a Cloud environment, `dbt retry` accepts only a small subset of overrides — typically `--threads`, `--vars`, and a few others. Run `dbt retry --help` locally for your CLI's exact list.

How it works under the hood

- Reads `run_results.json` to identify each node's last status.
- Builds the *remaining* DAG: failed nodes + their previously-skipped descendants.
- Runs that subset, **idempotently** — given the same code and same data, repeated retries (without fixing the bug) produce the same failure each time.

When retry won't help

If the previous run failed **before** any node executed — for example, a warehouse connection or permission error — there are no recorded nodes for retry to use. dbt's recommendation: inspect `run_results.json`, manually re-run the full job, and once nodes have started executing, retry becomes useful.

Example flow

Initial run with a syntax error

```
1 of 5 OK   created sql view model stg_customers
2 of 5 OK   created sql view model stg_orders
3 of 5 OK   created sql view model stg_payments
4 of 5 ERROR creating sql table model customers      <-- syntax error
5 of 5 OK   created sql table model orders

Completed with 1 error. PASS=4 ERROR=1 SKIP=0 TOTAL=5
```

Retry without fixing — still fails

```
1 of 1 ERROR creating sql table model customers
```

Idempotent — same input, same output.

Fix the SQL, then retry

```
1 of 1 OK   created sql table model customers
Done. PASS=1 ERROR=0 TOTAL=1
```

Only the failing node ran. Everything upstream/successful was skipped.

How retry pairs with microbatch

The microbatch incremental strategy makes each time-window batch its own retryable node. `dbt retry` reruns just the **failed batches**, not the whole model — a big win for big time-series tables.

Common patterns

```
# Standard retry of the prior invocation
dbt retry

# Retry but bump threads
dbt retry --threads 16

# Retry with overridden vars
dbt retry --vars '{"backfill_date": "2024-01-15"}'

# Force incrementals to rebuild in the retry
dbt retry --full-refresh

# Retry reading state from a CI artifact path
dbt retry --state path/to/saved-target
```

Where retry fits in the resilience story (CP3)

- A scheduled `dbt build` fails partway through.
- Operator gets paged.
- After diagnosing & fixing (probably with `dbt show` + `target/run/`), `dbt retry` finishes the job — no recomputing everything that already succeeded.
- Cheaper than a full rerun; faster mean-time-to-recovery.

Microbatch + retry is the canonical "resilient pipeline at scale" pattern: long time-series rebuilds are split into batches, failures retry per-batch.

Key takeaways

- `dbt retry` resumes the prior invocation from where it failed, using `run_results.json`.
- Idempotent — same inputs, same outcome, until you fix something.
- Inherits original selection on Core; Fusion lets you override `--select`/`--exclude`.
- Works with: `build`, `compile`, `clone`, `docs generate`, `seed`, `snapshot`, `test`, `run`, `run-operation`.
- If the prior run failed pre-node-execution, retry has nothing to resume — rerun the full job.
- Combine with microbatch for per-batch retry on huge time-series models.

Related

- `run_results.json` artifact.
- Microbatch incremental models (per-batch retry).
- `result:` selectors — the more flexible "pick up failures" alternative.
- `--state` flag.